SugaiAgent: An agent submitted to the ANAC 2026 SCM league

Toshiki Sugai[1] ,Takanobu Otsuka[2]
[1,2]Nagoya Institute of Technology, Aichi, Japan
[1]sugai.toshiki@otsukalab.nitech.ac.jp
[2]otsuka.takanobu@nitech.ac.jp

June 22, 2026

Abstract

SugaiAgent is a negotiation agent for the standard (Std) track of the SCM league. It handles negotiations with suppliers and consumers concurrently within a single synchronized loop, and is built around three ideas: *trust-weighted quantity allocation* across partners, *time-dependent price concession*, and *greedy acceptance that never exceeds the needed quantity* (over-contracting prevention). Across five tournament runs, SugaiAgent ranked first in every run, beating GreedyStdAgent by about 19% and SyncRandomStdAgent by about 36% in mean score. This report describes the design rationale, the implementation of concurrent negotiation and risk management, and the evaluation results.

1 Introduction

In the Std track of the SCM league, an agent acts as a middle-level trader that buys raw materials from suppliers and sells products to consumers at the same time. Profit is maximized by securing the needed quantity, signing contracts at favorable prices, and *avoiding the penalties of disposal (leftover stock) and shortfall (unmet demand)*. A strong agent therefore needs, simultaneously, (i) a pricing strategy that extracts favorable terms from each partner, (ii) control that drives many concurrent negotiations, and (iii) risk management so that signed contracts do not exceed the needed quantity.

SugaiAgent is implemented as a synchronized agent extending `StdSyncAgent`, deciding the responses to all partners together in each negotiation round. At the start of every step (`before_step`) it reads the tradable price range and the needed quantities, and uses them to propose, counter, and accept. The following sections describe the design.

2 The Design of the Agent

The overall flow is shown in Figure **??**. In each step, partners are split into a selling side (consumers) and a buying side (suppliers), and each side is processed independently according to its own need (`needed_sales` / `needed_supplies`).

2.1 Negotiation Choices

Normalized price utility. The desirability of a price is normalized to $[0, 1]$ per direction (`_price_utility`). On the selling side a higher price is better, on the buying side a lower price is better; the value is mapped linearly within that day's price range $[\, p_{\min}, p_{\max} \,]$. This lets buying and selling be compared in a single framework.

Opening price. The agent opens from its own favorable end and concedes only slightly (`_opening_price`). The concession width grows with the trust τ as $0.1 + 0.2\tau$, so it makes a small move toward partners with a high record of agreement.

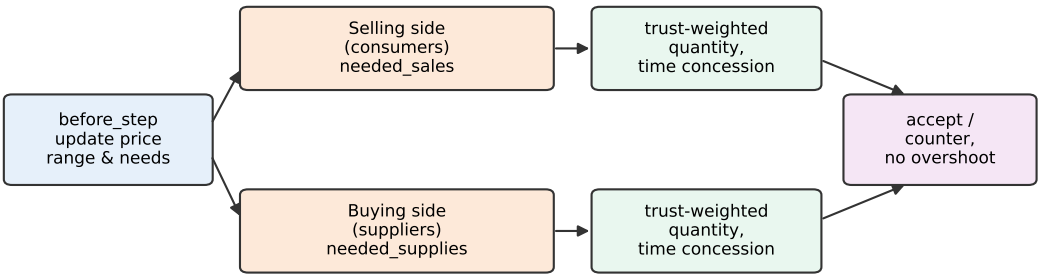

Figure 1: Two-sided synchronized negotiation in a single step. Each side performs trust-weighted quantity allocation and time-dependent price concession, and acceptance is capped so as not to exceed the needed quantity.

Time-dependent concession. The target price moves toward the unfavorable end as relative time $t \in [0, 1]$ advances (`_target_price`, concession factor $0.1 + 0.7t$). The counter price is the convex combination of the partner's offered price and the agent's own target,

$$p \;=\; \alpha\, p_{\text{offer}} + (1-\alpha)\, p_{\text{target}}, \qquad \alpha = \begin{cases} 0.8 & (t > 0.8) \\ 0.3 + 0.5\tau & (\text{otherwise}) \end{cases}$$

(`_counter_price`). The later the round or the more trusted the partner, the more weight is put on the partner's offered price, converging to agreement faster.

Acceptance decision. The absolute criterion for acceptance is a time-relaxed threshold $\theta = 0.6\,(1-t)$ on the normalized price utility. Early on, only high-profit offers are accepted; later the threshold is lowered to prioritize securing the needed quantity. The ranking itself uses the utility function `ufun`, and offers whose price utility exceeds the threshold are selected from the top down.

2.2 Concurrent Negotiation

Under the `StdSyncAgent` framework, `first_proposals` and `counter_all` return the responses to all partners simultaneously. SugaiAgent divides partners into the selling and buying sides and processes each independently in `_counter_side`.

First proposals. The needed quantity is allocated across partners weighted by trust. The proposed quantity for partner p blends the trust-based share $\text{need} \cdot \tau_p / \sum_q \tau_q$ with the historical average contract quantity `_avg_quantity` in equal parts, $q_p = \text{round}\big(0.5\,\text{need}\,w_p + 0.5\,\bar{q}_p\big)$, capped by the need. This assigns more to partners that tend to agree and less to new partners.

Counters and acceptance. `counter_all` ranks the incoming offers by utility and greedily accepts those above the threshold θ *without exceeding the need* (secured + $q \le$ need). For the remaining need, it counters by requesting the smaller of the remaining quantity and the partner's offered quantity, while respecting the partner's desired delivery time (`TIME`). It always returns a response to every partner and returns `None` (end negotiation) on the unnecessary side to avoid missing responses.

2.3 Risk Management

Over-contracting prevention. This is the most important design point of the agent. On a successful agreement (`on_negotiation_success`) the need on that side is decremented by the contracted quantity, and negotiations on a side whose need reaches zero or below are ended immediately. In addition, the acceptance stage caps the cumulative quantity at the need, and the counter quantity is bounded by the remaining need, so leftover stock and over-buying are structurally unlikely.

Trust estimation. The agreement probability per partner is estimated with Laplace smoothing, $\tau_p = (s_p + 1)/(t_p + 2)$ (with s successes and t trials). An unknown partner starts from 0.5, and division by zero is avoided.

Early termination of unpromising negotiations. A partner with more than 5 trials, trust below 0.2, and a negotiation already in its second half ($t > 0.5$) is dropped, redirecting the limited negotiation steps toward more promising partners.

Robustness. The price range and needs are refreshed every step in `before_step`, and the retrieval of price and delivery time falls back gracefully on exceptions (`_safe_price`, `_delivery_time`), so the agent runs without crashing even when issues are missing.

3 Evaluation

The evaluation used the setting `n_steps`= 50, `n_configs`= 5, `n_runs_per_world`= 1, running five tournaments against GreedyStdAgent and SyncRandomStdAgent (15 or 30 worlds, 45 or 60 scores). The score of each run is shown in Table **??** and Figure **??**.

Table 1: Score per run (worlds in parentheses) and mean.

Agent	Run1 (15)	Run2 (30)	Run3 (15)	Run4 (30)	Run5 (15)	**Mean**
SugaiAgent	**1.0839**	**1.0503**	**0.9066**	**1.0711**	**1.0424**	**1.0308**
GreedyStdAgent	0.9674	0.7726	0.8698	0.9136	0.8068	0.8660
SyncRandomStdAgent	0.7754	0.6216	0.7346	0.9316	0.7237	0.7574

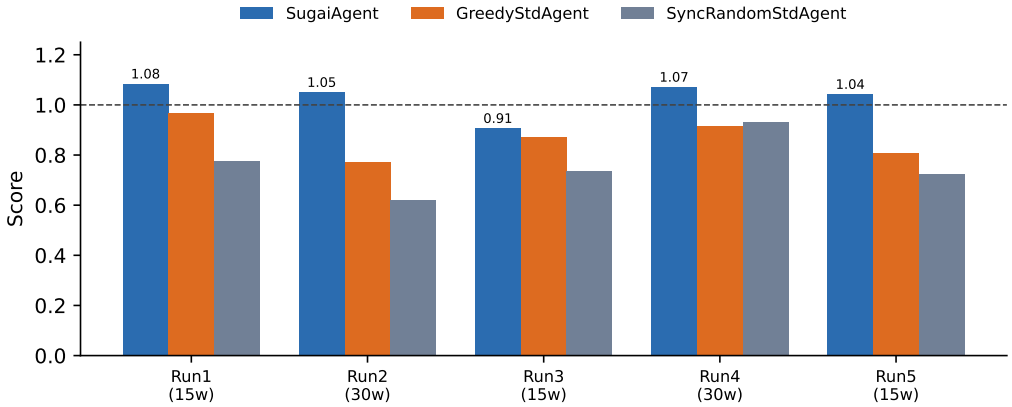

Figure 2: Score comparison per run (dashed line at score 1.0). SugaiAgent is top in all five runs.

SugaiAgent ranked **first in all five runs**, with a mean score of 1.031. This is about **19%** above GreedyStdAgent (mean 0.866) and about **36%** above SyncRandomStdAgent (mean 0.757). In Run3 the gap narrowed because there were fewer worlds, but the ranking held, and even in Run4—where the order of the two baselines swapped—the agent kept a stable advantage across opponent compositions. This stability is plausibly due to the over-contracting prevention and the strict tracking of the needed quantity, which suppress the disposal/shortfall penalties.

4 Lessons and Suggestions

The evaluation confirms that simple rules—acceptance that never exceeds the need and trust-weighted quantity allocation—yield a stable advantage over the naive Greedy strategy. There is, however, room for improvement. First, trust is defined solely by agreement probability, so the *quality of the agreed price* is not reflected; evaluating partners by the product of price utility and agreement probability (expected utility) could reduce the share of low-margin agreements. Second, the need

relies on the per-day value returned by the AWI and does not *look ahead* to future-step demand or to inventory and exogenous-contract dynamics. Third, fixing the delivery time to its earliest value underuses the available scheduling freedom.

As suggestions: (i) learn each partner's concession curve to adapt the counter weight α; (ii) dynamically adjust the acceptance threshold θ to the competitive density of the market; and (iii) introduce a quantity plan that anticipates supply and demand several steps ahead. These three directions appear most promising.

Conclusions

SugaiAgent is a negotiation agent that combines trust-weighted quantity allocation, time-dependent price concession, and greedy acceptance that never exceeds the need, on top of synchronized two-sided concurrent negotiation. By structurally preventing over-contracting, it suppresses disposal/shortfall penalties and consistently beat GreedyStdAgent and SyncRandomStdAgent over five tournaments (by about 19% and 36% in the mean, respectively). Future work aims to further improve profit through expected-utility-based partner evaluation and look-ahead demand planning.